DBT Assignment 5

1. Check Even or Odd

Create a stored procedure check_even_odd that accepts an integer and displays whether the number is Even or Odd.

Test cases:

CALL check_even_odd(25);

CALL check_even_odd(40);

```
mysql> delimiter //
mysql> create procedure check_even_odd(in num int)
    -> begin
    -> if num % 2 = 0 then
    -> select 'Even' as result;
    -> else
    -> select 'Odd' as result;
    -> end if;
    -> end //
Query OK, 0 rows affected (0.06 sec)

mysql> delimiter ;
mysql> call check_even_odd(25);
+--------+
| result |
+--------+
| Odd    |
+--------+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.03 sec)

mysql> call check_even_odd(40);
+--------+
| result |
+--------+
| Even   |
+--------+
1 row in set (0.00 sec)
```

2. Calculate Simple Interest

Create a stored procedure calculate_interest that accepts principal amount, rate of interest, and time in years and calculates Simple Interest.

Simple Interest = (Principal × Rate × Time) / 100

Test cases:

CALL calculate_interest(10000, 5, 2);

CALL calculate_interest(25000, 7, 3);

```
mysql> delimiter //
mysql> create procedure calculate_interest(in si_p decimal(10, 2), si_r decimal(5, 2), si_n int)
    -> begin
    -> declare simple_interest decimal(10, 2);
    -> set simple_interest = (si_p * si_r * si_n)/100;
    -> select simple_interest;
    -> end //
Query OK, 0 rows affected (0.04 sec)

mysql> delimiter ;
mysql> call calculate_interest(10000, 5, 2);
+-----------------+
| simple_interest |
+-----------------+
|         1000.00 |
+-----------------+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> call calculate_interest(25000, 7, 3);
+-----------------+
| simple_interest |
+-----------------+
|         5250.00 |
+-----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

3. Calculate Grade

Create a stored procedure calculate_grade that accepts student ID, student name, and marks and displays the student's grade.

Marks	Grade
90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F

Test cases:

CALL calculate_grade(101, 'Amit', 85);

CALL calculate_grade(102, 'Priya', 72);

CALL calculate_grade(103, 'Rahul', 45);

```
mysql> delimiter //
mysql> create procedure calculate_grade(in student_id int, student_name varchar(40), marks int)
    ->
    -> begin
    -> declare grades varchar(2);
    ->
    -> if marks>=90 and marks<=100 then
    -> set grades = 'A+';
    ->
    -> elseif marks>=80 then
    -> set grades = 'A';
    ->
    -> elseif marks>=70 then
    -> set grades = 'B';
    ->
    -> elseif marks>=60 then
    -> set grades = 'C';
    ->
    -> elseif marks>=50 then
    -> set grades = 'D';
    ->
    -> else
    -> set grades = 'F';
    ->
    -> end if;
    ->
    -> select student_id, student_name, marks, grades;
    ->
    -> end //
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> delimiter ;
mysql> call calculate_grade(101, 'Amit', 85);
+------------+--------------+-------+--------+
| student_id | student_name | marks | grades |
+------------+--------------+-------+--------+
|        101 | Amit         |    85 | A      |
+------------+--------------+-------+--------+
1 row in set (0.05 sec)

Query OK, 0 rows affected (0.06 sec)

mysql> call calculate_grade(102, 'Priya', 72);
+------------+--------------+-------+--------+
| student_id | student_name | marks | grades |
+------------+--------------+-------+--------+
|        102 | Priya        |    72 | B      |
+------------+--------------+-------+--------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> call calculate_grade(103, 'Rahul', 45);
+------------+--------------+-------+--------+
| student_id | student_name | marks | grades |
+------------+--------------+-------+--------+
|        103 | Rahul        |    45 | F      |
+------------+--------------+-------+--------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)
```

4. Calculate Employee Bonus

Create a stored procedure calculate_bonus that accepts employee name and salary and calculates the bonus according to the following rules. Display employee name, salary, bonus percentage, bonus amount, and total salary.

Salary	Bonus
≥ 100000	20%
≥ 50000	15%
≥ 30000	10%
Below 30000	5%

Test cases:

CALL calculate_bonus('Amit', 120000);

CALL calculate_bonus('Priya', 65000);

CALL calculate_bonus('Rahul', 25000);

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE calculate_bonus(
    ->     IN emp_name VARCHAR(40),
    ->     IN salary DECIMAL(10,2)
    -> )
    -> BEGIN
    ->     DECLARE bonus_per INT;
    ->     DECLARE bonus_amt DECIMAL(10,2);
    ->     DECLARE total_sal DECIMAL(10,2);
    ->
    ->     IF salary >= 100000 THEN
    ->         SET bonus_per = 20;
    ->     ELSEIF salary >= 50000 THEN
    ->         SET bonus_per = 15;
    ->     ELSEIF salary >= 30000 THEN
    ->         SET bonus_per = 10;
    ->     ELSE
    ->         SET bonus_per = 5;
    ->     END IF;
    ->
    ->     SET bonus_amt = salary * bonus_per / 100;
    ->     SET total_sal = salary + bonus_amt;
    ->
    ->     SELECT emp_name AS Name,
    ->            salary AS Emp_salary,
    ->            CONCAT(bonus_per, '%') AS bonus_percentage,
    ->            bonus_amt AS bonus_amount,
    ->            total_sal AS total_salary;
    -> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
mysql> CALL calculate_bonus('Amit', 120000);
+------+------------+------------------+--------------+--------------+
| Name | Emp_salary | bonus_percentage | bonus_amount | total_salary |
+------+------------+------------------+--------------+--------------+
| Amit |  120000.00 | 20%              |     24000.00 |    144000.00 |
+------+------------+------------------+--------------+--------------+
1 row in set (0.03 sec)

Query OK, 0 rows affected (0.04 sec)

mysql> CALL calculate_bonus('Priya', 65000);
+-------+------------+------------------+--------------+--------------+
| Name  | Emp_salary | bonus_percentage | bonus_amount | total_salary |
+-------+------------+------------------+--------------+--------------+
| Priya |   65000.00 | 15%              |      9750.00 |     74750.00 |
+-------+------------+------------------+--------------+--------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> CALL calculate_bonus('Rahul', 25000);
+-------+------------+------------------+--------------+--------------+
| Name  | Emp_salary | bonus_percentage | bonus_amount | total_salary |
+-------+------------+------------------+--------------+--------------+
| Rahul |   25000.00 | 5%               |      1250.00 |     26250.00 |
+-------+------------+------------------+--------------+--------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)
```

5. Find Maximum of Three Numbers —

Create a stored procedure find_maximum that accepts three integers and displays the largest number.

Test cases:

CALL find_maximum(25, 78, 45);

CALL find_maximum(100, 50, 75);

CALL find_maximum(30, 30, 20);

```
mysql> delimiter //
mysql> create procedure find_maximum(in num1 int, num2 int, num3 int)
    ->
    -> begin
    ->
    -> if num1 >= num2 and num1 >= num3 then
    -> select num1 as largest_number;
    ->
    -> elseif num2 >= num1 and num2 >= num3 then
    -> select num2 as largest_number;
    ->
    -> else
    -> select num3 as largest_number;
    ->
    -> end if;
    ->
    -> end //
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> delimiter ;
mysql> call find_maximum(25, 78, 45);
+----------------+
| largest_number |
+----------------+
|             78 |
+----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> call find_maximum(100, 50, 75);
+----------------+
| largest_number |
+----------------+
|            100 |
+----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> call find_maximum(30, 30, 20);
+----------------+
| largest_number |
+----------------+
|             30 |
+----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

6. Employee Salary Analysis

Create a stored procedure salary_analysis that accepts employee name and salary. Determine the salary category and allowance, then calculate the final salary. Display employee name, salary, category, allowance percentage, allowance amount, and final salary.

Salary	Category	Allowance
≥ 100000	High	20%
≥ 50000	Medium	15%
Below 50000	Low	10%

Allowance = Salary × Allowance Percentage / 100

Final Salary = Salary + Allowance

Test cases:

CALL salary_analysis('Amit', 120000);

CALL salary_analysis('Priya', 60000);

CALL salary_analysis('Rahul', 35000);

```
mysql> delimiter //
mysql> create procedure salary_analysis(in emp_name varchar(50), emp_sal decimal(10,2))
    ->
    -> begin
    -> declare emp_category varchar(10);
    -> declare allow_per int;
    -> declare allow_amt decimal(10,2);
    -> declare final_sal decimal(10,2);
    ->
    -> if emp_sal >= 100000 then
    -> set emp_category = 'High';
    -> set allow_per = 20;
    ->
    -> elseif emp_sal >= 50000 then
    -> set emp_category = 'Medium';
    -> set allow_per = 15;
    ->
    -> else
    -> set emp_category = 'Low';
    -> set allow_per = 10;
    ->
    -> end if;
    ->
    -> set allow_amt = emp_sal * allow_per / 100;
    -> set final_sal = emp_sal + allow_amt;
    ->
    -> select emp_name as Name,
    -> emp_sal as salary,
    -> emp_category as Category,
    -> concat(allow_per, '%') as Allowance_percentage,
    -> allow_amt as Allowance_amount,
    -> final_sal as Final_Salary;
    ->
    -> end //
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> delimiter ;
mysql> call salary_analysis('Amit', 120000);
+------+-----------+----------+----------------------+------------------+--------------+
| Name | salary    | Category | Allowance_percentage | Allowance_amount | Final_Salary |
+------+-----------+----------+----------------------+------------------+--------------+
| Amit | 120000.00 | High     | 20%                  |         24000.00 |    144000.00 |
+------+-----------+----------+----------------------+------------------+--------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.03 sec)

mysql> CALL salary_analysis('Priya', 60000);
+-------+----------+----------+----------------------+------------------+--------------+
| Name  | salary   | Category | Allowance_percentage | Allowance_amount | Final_Salary |
+-------+----------+----------+----------------------+------------------+--------------+
| Priya | 60000.00 | Medium   | 15%                  |          9000.00 |     69000.00 |
+-------+----------+----------+----------------------+------------------+--------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> CALL salary_analysis('Rahul', 35000);
+-------+----------+----------+----------------------+------------------+--------------+
| Name  | salary   | Category | Allowance_percentage | Allowance_amount | Final_Salary |
+-------+----------+----------+----------------------+------------------+--------------+
| Rahul | 35000.00 | Low      | 10%                  |          3500.00 |     38500.00 |
+-------+----------+----------+----------------------+------------------+--------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)
```

7. Electricity Bill Calculation

Create a stored procedure calculate_electricity_bill that accepts consumer name and units consumed. Calculate the electricity bill using the slab-wise rates below. Display consumer name, units consumed, and total bill amount.

Units	Rate
First 100 units	₹2 per unit
Next 100 units	₹3 per unit
Next 200 units	₹5 per unit
Above 400 units	₹7 per unit

Test cases:

CALL calculate_electricity_bill('Amit', 80);

CALL calculate_electricity_bill('Priya', 150);

CALL calculate_electricity_bill('Rahul', 350);

CALL calculate_electricity_bill('Sneha', 500);

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE calculate_electricity_bill(
    ->     IN consumer_name VARCHAR(40),
    ->     IN units INT
    -> )
    -> BEGIN
    ->     DECLARE bill_amount DECIMAL(10,2);
    ->
    ->     IF units <= 100 THEN
    ->         SET bill_amount = units * 2;
    ->
    ->     ELSEIF units <= 200 THEN
    ->         SET bill_amount = (100 * 2)
    ->                         + ((units - 100) * 3);
    ->
    ->     ELSEIF units <= 400 THEN
    ->         SET bill_amount = (100 * 2)
    ->                         + (100 * 3)
    ->                         + ((units - 200) * 5);
    ->
    ->     ELSE
    ->         SET bill_amount = (100 * 2)
    ->                         + (100 * 3)
    ->                         + (200 * 5)
    ->                         + ((units - 400) * 7);
    ->     END IF;
    ->
    ->     SELECT consumer_name AS Consumer_Name,
    ->            units AS Units_Consumed,
    ->            bill_amount AS Total_Bill_Amount;
    -> END //
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> DELIMITER ;
mysql> CALL calculate_electricity_bill('Amit', 80);
+---------------+----------------+-------------------+
| Consumer_Name | Units_Consumed | Total_Bill_Amount |
+---------------+----------------+-------------------+
| Amit          |             80 |            160.00 |
+---------------+----------------+-------------------+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> CALL calculate_electricity_bill('Priya', 150);
+---------------+----------------+-------------------+
| Consumer_Name | Units_Consumed | Total_Bill_Amount |
+---------------+----------------+-------------------+
| Priya         |            150 |            350.00 |
+---------------+----------------+-------------------+
1 row in set (0.02 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> CALL calculate_electricity_bill('Rahul', 350);
+---------------+----------------+-------------------+
| Consumer_Name | Units_Consumed | Total_Bill_Amount |
+---------------+----------------+-------------------+
| Rahul         |            350 |           1250.00 |
+---------------+----------------+-------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> CALL calculate_electricity_bill('Sneha', 500);
+---------------+----------------+-------------------+
| Consumer_Name | Units_Consumed | Total_Bill_Amount |
+---------------+----------------+-------------------+
| Sneha         |            500 |           2200.00 |
+---------------+----------------+-------------------+
1 row in set (0.00 sec)
```